

L1-Encapsulation and Polymorphism

1.1. What is encapsulation?

Encapsulation means locking data inside a class and only allowing access through controlled method.

We mark fields private so no one outside the class can touch them directly. If they want to read or change the value, they must go through our public methods, where I can add checks. Real-life analogy :-

A medicine capsule; The medicine (data) is sealed inside a shell. We cannot touch the medicine directly, we only interact with the capsule as a whole (the public interface).

1.2. What is polymorphism?

Polymorphism means "one name only, many forms". The same method name behaves differently depending on the situation.

▣ Compile time polymorphism:— (Static)

It refers to method overloading. The compiler decides which method to run based on the parameters, before the program runs.

Example:— `add(int a, int b)` vs `add(double a, double b)`.

▣ Runtime polymorphism:— (dynamic)

It refers to method overriding. The JVM decides which method to run while the program is actually running, based on the real object

type. Example:— `Animal a = new Dog();`
`a.sound();`

1.3. Practical :- Bank account class:-

Code:-

```
class BankAccount {  
    private double balance;  
    public double getBalance() {  
        return balance; }  
    // Overload 1  
    public void deposit (double amount) {  
        balance += amount;  
        System.out.println("Deposited: "+  
                             amount);  
    // Overload 2 (compile time)  
    public void deposit(double amount, String remarks) {  
        balance += amount;  
        System.out.println("Deposited: " + amount + "  
                             Remarks: " + remarks);  
    }  
}  
Public class Main {  
    public static void main (String[] args) {  
        BankAccount acc = new BankAccount();  
        acc.deposit(500);  
        acc.deposit(1000, "Salary credit");  
        System.out.println("Balance: " + acc.getBalance());  
    }  
}
```

L2- Method overloading and Overriding

2.1. Method Overloading vs Overriding

Overloading	Overriding
Same method name but, different parameters	Same method name & parameter but redefined in a subclass
Happend in same class	In parent and child class
Parameter must be different	Must be same
Return type can different	Must be same or a subtype
Compile time, early binding	Run time, late binding

2.2. Explain early vs late binding. Why is overriding resolved at run time and overloading at compile time?

Early binding:

The compiler already knows exactly which method to call, just by looking at the method. This happens with overloading, static, private and final method.

Late binding:

The compiler doesn't know the final method to call in advance - the JVM checks the actual object at run time and calls the right version. This happens with overriding. Overriding needs late binding because the same reference type can point to different objects. Java can't know which one until running.

2.3 - Practical : Shape hierarchy : —

```
class Shape {  
    public double area () {  
        return 0;  
    }  
}
```

// Overloading (early binding)

```
public void describe (String name) {  
    System.out.println ("Shape : "+ name);  
}
```

```
public void describe (String name, int sides) {  
    System.out.println ("Shape : "+ name + ", sides : "  
        + sides);  
}
```

```
class Circle extends Shape {  
    private double radius;  
    public Circle (double radius) {  
        this.radius = radius;  
    }  
}
```

@ Override

```
public double area () { return Math.PI * radius *  
    radius; }  
}
```

```

class Rectangle extends Shape {
    private double length, width;
    public Rectangle (double length, double
                                width) {
        this.length = length;
        this.width = width; }

```

```

    @Override
    public double area() { return length * width; }
}

```

```

Public class Main {
    public static void main (String[] args) {
        Shape s1 = new Circle(5); // late binding
        Shape s2 = new Rectangle(4, 5);
        System.out.println("Circle area:" + s1.area());
        System.out.println("Rectangle area:" + s2.area());
        s1.describe("Circle");
        s2.describe("Rectangle", 4); // Early binding
    }
}

```

L3- Abstract Class vs Interface

3.1- Definition and difference

- Abstract class: A class that cannot be instantiated directly. It can mix finished methods with unfinished methods that subclasses must complete.
- Interface: A contract listing method signatures with no body. Any class that implements it must fill in those methods.

Abstract	Interface
can have normal instance variable	Only public static final constants
Can have one constructor	NO constructor
Mix of implemented method bodies	Only default/static methods bodies
A class can extend only one abstract class	A class can have many interface implemented

3.2- When should we choose abstract class vs an interface ?

Abstract class:

Use an abstract class for an "is-a" relationship where subclasses share common code.

Example:- Vehicle is a base for car, Bike — they all share StartEngine() logic.

Interface:

Use Interface for a "can-do" capability that unrelated classes might share.

Example:- Insurable — a car, a house, or even a person ; could all be insurable, even though they're not related by inheritance.

3.3- Practical: Vehicle / Insurable / car

code:

```
abstract class Vehicle {  
    public void startEngine() {  
        System.out.println("Engine started");  
    }  
    public abstract String fuelType();  
}
```

```
interface Insurable {  
    double calculatePremium();  
}
```

```
class car extends Vehicle implements Insurable {  
    @Override  
    public String fuelType() {  
        return "Petrol";  
    }  
}
```

@Override

```
public double calculatePremium() {  
    return 15000.0;  
}
```

```
public class Main {  
    public static void main (String [] args) {  
        Car car = new Car();  
        car.startEngine();  
        // inherited concrete method  
        System.out.println("Fuel:" + car.fuelType());  
        System.out.println("Premium:" + car.calculate  
            premium());  
    }  
}
```

L4- Collection Framework

4.1 - ArrayList, Vector vs Linked List

ArrayList	Vector	Linked List
Resizable array	Resizable array	Double linked List
Not synchronized	Synchronized	Not synchronized
Fast random access	Fast random access.	Slow random access
Insert/Delete Slow	Slow	Fast

4.2 - Explain Set and its implementations

A set is a collection that does not allow duplicate elements.

- i) HashSet: no guaranteed Order, fastest.
- ii) LinkedHashSet: keeps insertion order.

iii) TreeSet: keeps elements sorted automatically. TreeSet maintains order using a Red-Black Tree internally - every time you add an element, it's placed in the correct sorted position

4.3- Practical: ArrayList vs TreeSet

Code:

```
import java.util. ArrayList;
import java.util. TreeSet;

public class Main {
    public class static void main (String[] args) {
        ArrayList< String> list = new ArrayList<> ();
        list.add ("David");
        list.add ("Alice");
        list.add ("Ethan");
    }
}
```

```
list.add("Bob");
```

```
list.add("Carol");
```

```
System.out.println("ArrayList (insertion  
order):");
```

```
for (String name: list) {
```

```
    System.out.println(name);
```

```
}
```

```
TreeSet<String> set = new TreeSet<>(list);
```

```
System.out.println("\nTree Set (sorted  
order):");
```

```
for (String name: set) {
```

```
    System.out.println(name);
```

```
}
```

```
}
```

```
}
```

Lab-5: Multithreading & Custom Exception

5.1: Explain ways to implement multithreading?

Am: Ways:—

- i) Extending Thread and overriding run().
- ii) Implementing Runnable and passing it to a Thread object.
- iii) Using ExecutorService / Callable — a thread pool manages threads for you, and callable can return a result. Preferred:—
Runnable is generally preferred, because java only allows single inheritance — if your class extends Thread, it can't extend anything else.

5.2 - Practical : two threads printing 1-5 :-

```
class MyThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread (extends): " + i);  
        }  
    }  
}
```

```
class MyRunnable implements Runnable {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread (Runnable): " + i);  
        }  
    }  
}
```

```
public class Main {  
    public static void main (String[] args) {  
        MyThread t1 = new MyThread();  
        Thread t2 = new Thread (new MyRunnable());  
        t1.start(); t2.start();  
    }  
}
```

5.3- Practical : custom checked exception

code:

```
class InvalidRadiusException extends Exception {  
    public InvalidRadiusException (String msg) {  
        super (msg);  
    }  
}
```

```
class circle {  
    private double radius;  
    public circle (double radius) throws InvalidRadiusException {  
        if (radius < 0) {  
            throw new InvalidRadiusException ("Radius cannot  
                be negative : " + radius);  
        }  
        this.radius = radius;  
    }  
    public double area () {  
        return Math.PI * radius * radius;  
    }  
}
```

```
import java.util.Scanner;
public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println ("Enter radius: ");
        double r = sc.nextDouble();
        try {
            circle c = new circle (r);
            System.out.println ("Area: " + c.area());
        }
        catch (InvalidRadiusException e) {
            System.out.println ("Error: " + e.getMessage());
        }
    }
}
```

Lab-6: JDBC with MySQL/Oracle

6.1 - Steps to connect Java to a database via JDBC

Ans:—

- i) Load/register the JDBC driver
- ii) Open a connection : `Connection con = DriverManager.getConnection(url, user, password);`
- iii) Create a statement : `PreparedStatement ps = con.prepareStatement(sql);`
- iv) Run the query/update : `ps.executeQuery()`
or `ps.executeUpdate()`.
- v) Process result using `ResultSet` (for SELECT queries).
- vi) Close the connection. Key classes: `DriverManager`, `Connection`, `PreparedStatement`, `ResultSet`.

6.2- Explain the MVC pattern and how a JDBC app's class map to Model/Controller.

Model - the plain data class representing one record.

View - Whatever collects input and shows output to the users

Controller - the class that handles logic and talks to the database, sitting between Model and View.

6.3 - Practical: MVC - Style Student Incent

```
import java.sql.*;
```

```
import java.util.*;
```

```
//Model
```

```
class Student {
```

```
    private int id;
```

```
    private String name;
```

```
    private double cgpa;
```

```
public Student (int id, String name, double gpa)
{
    this.id = id;
    this.name = name;
    this.gpa = gpa;
}
```

```
public int getID() { return id; }
public String getName() { return name; }
public double getGpa() { return gpa; }
```

```
}
public class Main {
    public static void main (String[] args)
        throws SQLException {
        Scanner sc = new Scanner (System.in);
        String name = sc.next();
        System.out.print ("CGPA: ");
        double gpa = sc.nextDouble();
        System.out.println ("Student saved!");
    }
}
```

Lab-7: JavaFX - House Loan Calculator

7.1- JavaFX application Structure

Application:-

The base class every JavaFX app extends. JavaFX calls its `start()` method to launch the GUI.

Stage:- The actual window on screen.

Scene:-

The content that fills the window.

GridPane/VBox:-

Layout containers that arrange buttons, labels and text fields. Role of `start()`: It's the entry point JavaFX calls automatically after launch.

7.2- Practical : GridPane layout

Code:-

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;

grid.setPadding(new Insets(10));
grid.setHgap(10);

private void calculate() {
    double p = Double.parseDouble(loanField.getText());
    resultLabel.setText(String.format(
        "Monthly : %.2f | Total : %.2f | Difference : %.2f",
        M, T, D));
}

public static void main(String[] args) {
    launch(args);
}
}
```

7.3 - Event handler computing Monthly installment, total payment, Difference payment.

Am:

$$\text{Annual, Rate} = \text{Annual Rate} / 12 / 100$$

Monthly installment,

$$M = P \times r \times (1+r)^n / [(1+r)^n - 1]$$

Time

$$T = M \times n$$

Difference,

$$D = T - P$$

T = total payment.

These handles all the events.

Lab-8: Socket Programming & Java RMI

8.1- Socket programming vs Java RMI

- Sockets:-

low-level, you send and receive raw text/bytes yourself over a socket/server socket connection. You control the exact message format.

- RMI (Remote Method Invocation):

High level, you call a method on a remote object almost like its local java handles the network communication for you. When to prefer each: Use sockets when you need full control over the protocol, ~~or~~ cross-language communication, use RMI when you're working purely in java and want to call remote method directly.

8.2 - Practical : Server code

```
import java.io.*;
import java.net.*;
public class chatServer {
    public static void main (String[] args)
        throws IOException {
        Socket socket = serverSocket.accept();
        System.out.println ("Client connected!");

        String message = in.readLine();
        System.out.println ("Client says: "+message);
        System.out.println ("Server received: "+message);
        socket.close();
        serverSocket.close();
    }
}
```

8.3 - Practical: Client code

```
import java.io.*;
import java.net.*;
public class chatclient {
    public static void main (String[] args)
        throws IOException {
        Socket socket = new Socket("local host",
                                   5000);
        PrintWriter out = new PrintWriter (
            socket.getOutputStream(), true);
        out.println ("Hello Server!");
        String reply = in.readLine();
        System.out.println ("Server reply: " + reply);
        socket.close();
    }
}
```

Lab-9: Servlet + JSP and JDBC CRUD (Records)

9.1- Steps set up the project

- i) Create the MySQL database Student-db and a Students table with columns id, name, cgpa.
- ii) Set up a Dynamic web project in IDE, add the MySQL JDBC connector .jar
- iii) Map the Servlet to a url pattern using @WebServlet or web.xml.
- iv) Build the JDBC connection string
- v) Write the Servlet (handle from Submission), a DAO class and a JSP page that shows result.

9.2- Practical: Servlet doPost().

```
import java.io.*;

String url = "jdbc:mysql://localhost:3306/
            student_db";

try (Connection con = DriverManager.getConnection
     (url, "root", "password");
     PreparedStatement ps = con.prepareStatement(
     "Insert into student (id, name, cgpa) values
     (?, ?, ?)")) {
    ps.setInt (1, id);
    ps.setString (2, name);
    ps.setDouble (3, cgpa);
    ps.executeUpdate();
    response.getWriter().println ("String added
    successfully!");
} catch (SQLException e) {
    response.getWriter().write("");
}
}
```

2.3- Practical : JSP page

```
<%@ page import = "java.sql.*"%>
<html>
<body>
<h2> Student List </h2>
<table border = "1">
<tr> <th> ID </th> Name </th> <th> CGPA </th>
<% connection con = DriverManager.getConnection(
    " jdbc:mysql://localhost:3306/Student_db",
    "root","password");
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * From
Students");
while (rs.next()) {
%>
<tr>
<td> <% = rs.getInt("id") %> </td>
<td> <% = rs.getString("name") %> </td>
<td> <% = rs.getDouble("cgpa") %> </td>
</tr>
<% } con.close(); %>
</table> </body>
</html>
```